Sprawozdanie-Algorytmy sortowania

Patryk Raduła i Jan Wójcik

24 marca 2026

1 Informacje wstępne

1.1 Język programowania

Wszystkie algorytmy sortowania zostały zaimplementowane w C. Generator danych wejściowych oraz program tworzący wykresy zostały zaimplementowane w Python.

1.2 Platforma na której wykonano testy

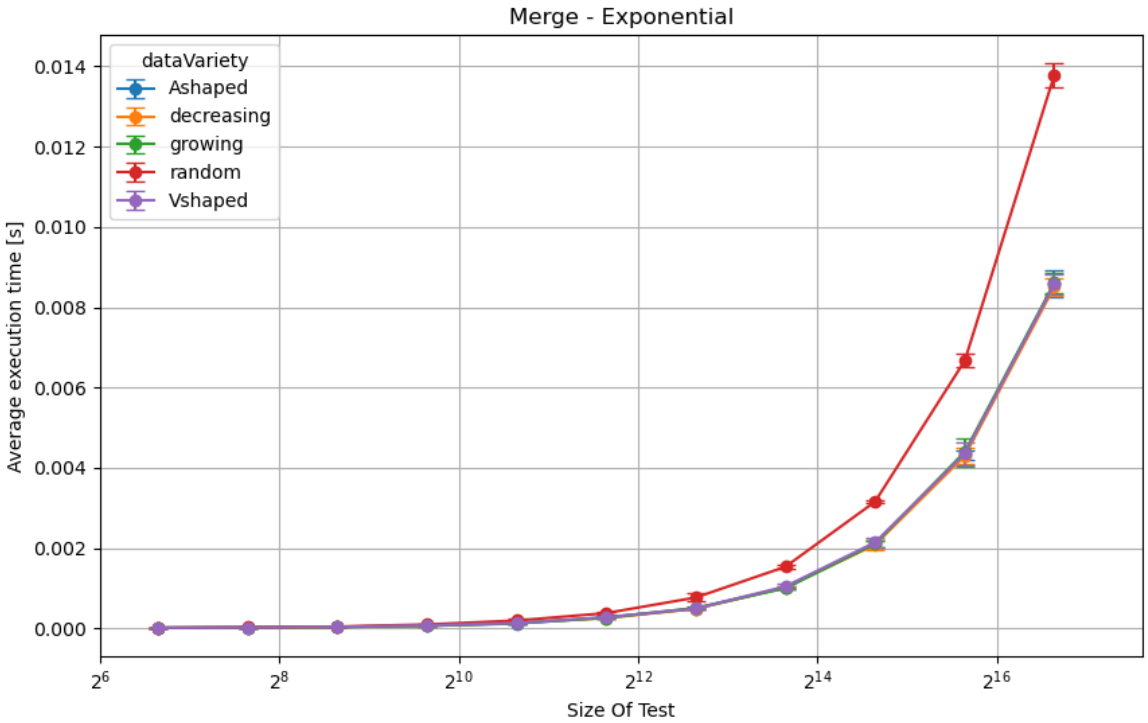
Testy przeprowadzono na komputerze osobistym o następującej specyfikacji:

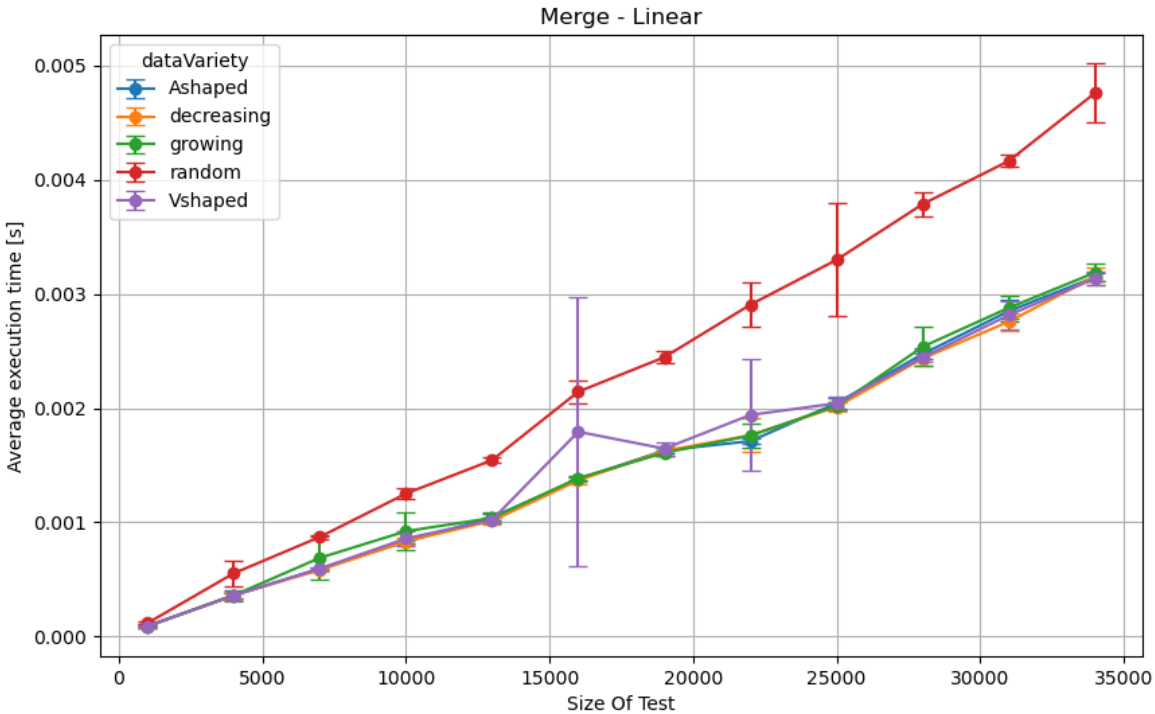
System operacyjny Windows 11 Home

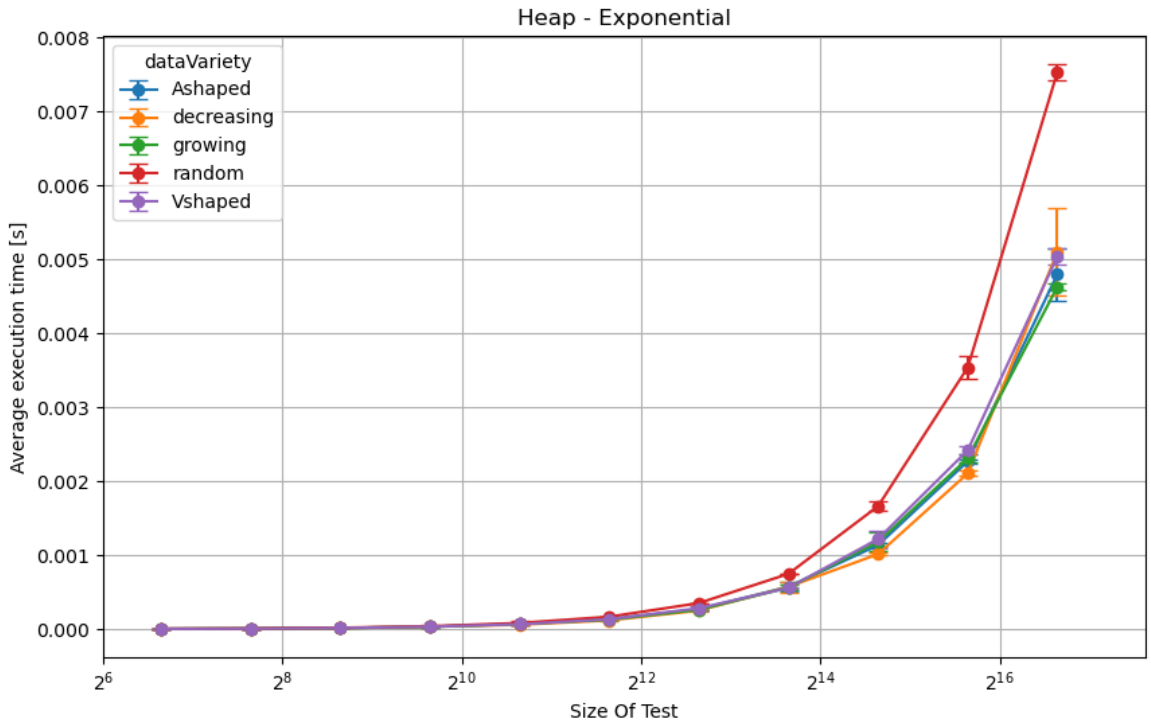
Procesor AMD Ryzen 5 7535HS with Radeon Graphics (3.30 GHz)

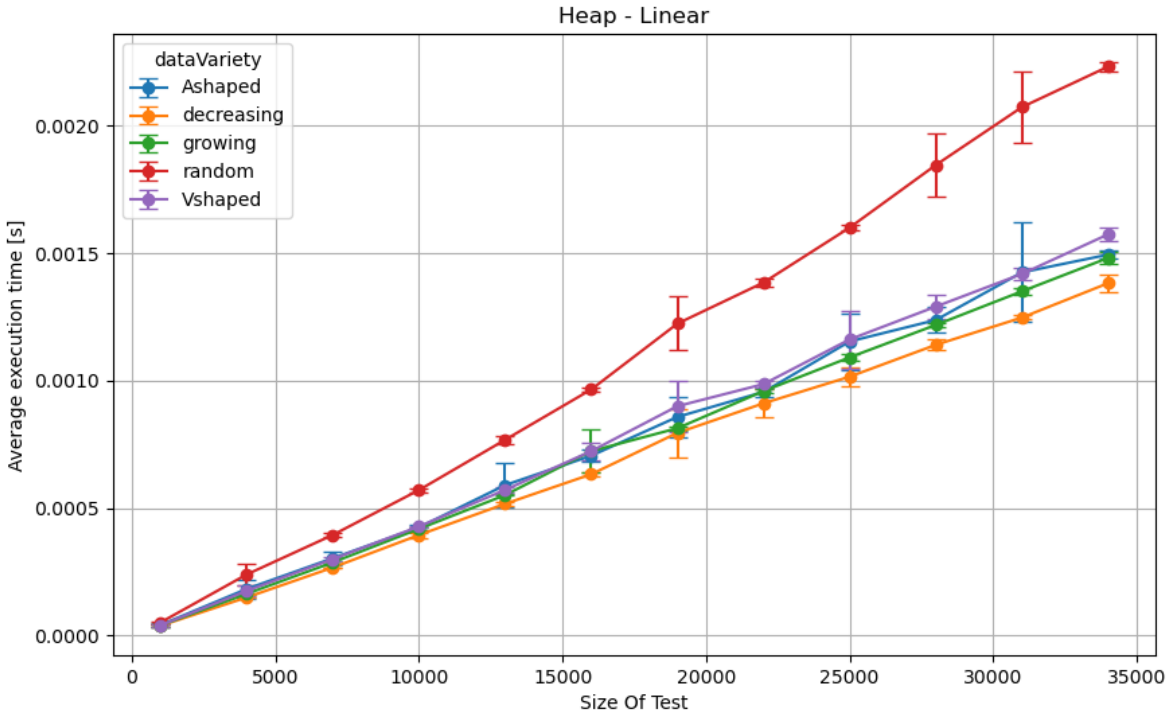
Pamięć RAM 16GB

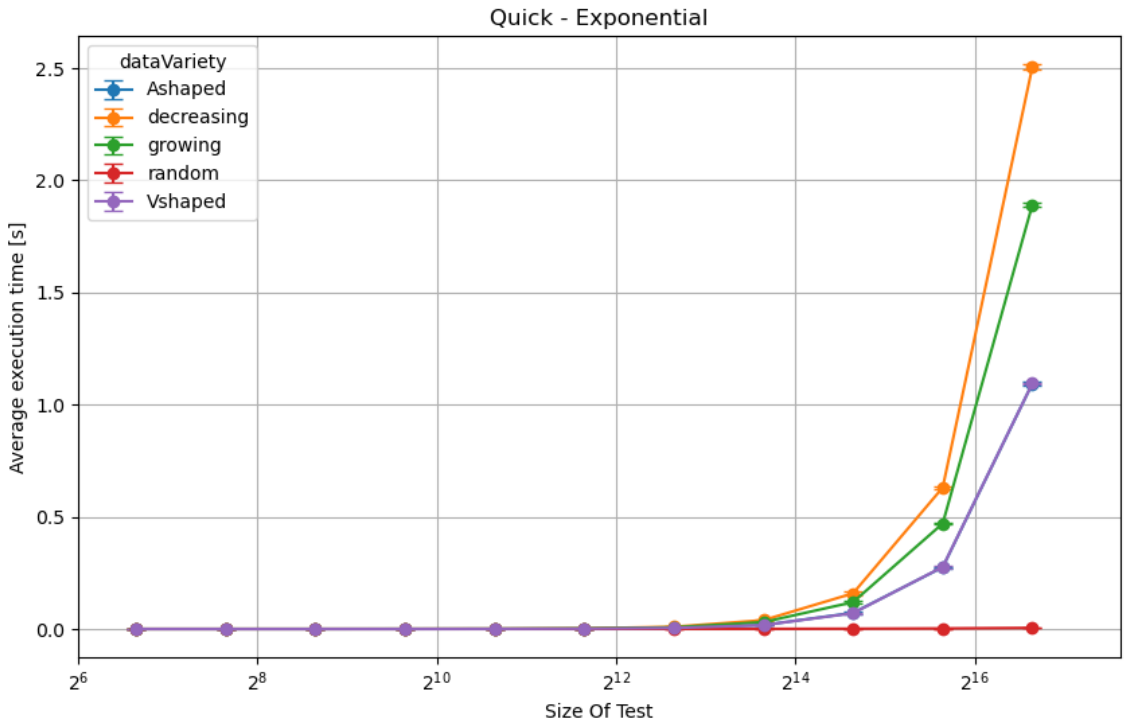
2 Wykresy zależności czasu od liczby elementów według algorytmu.

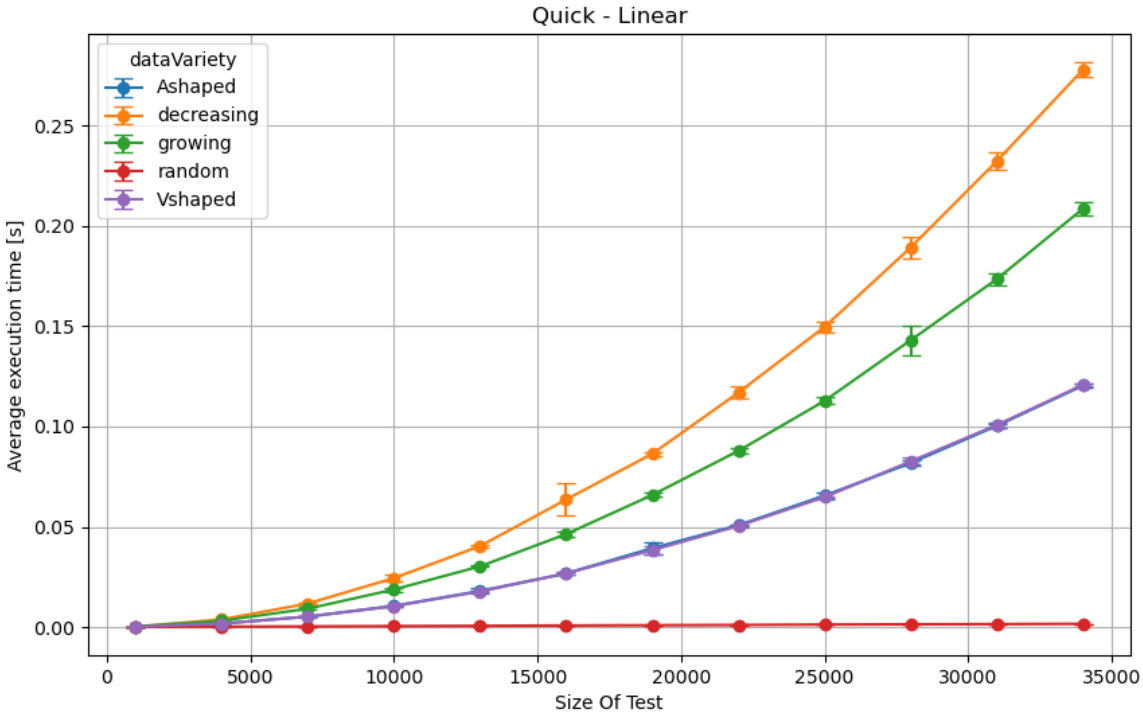


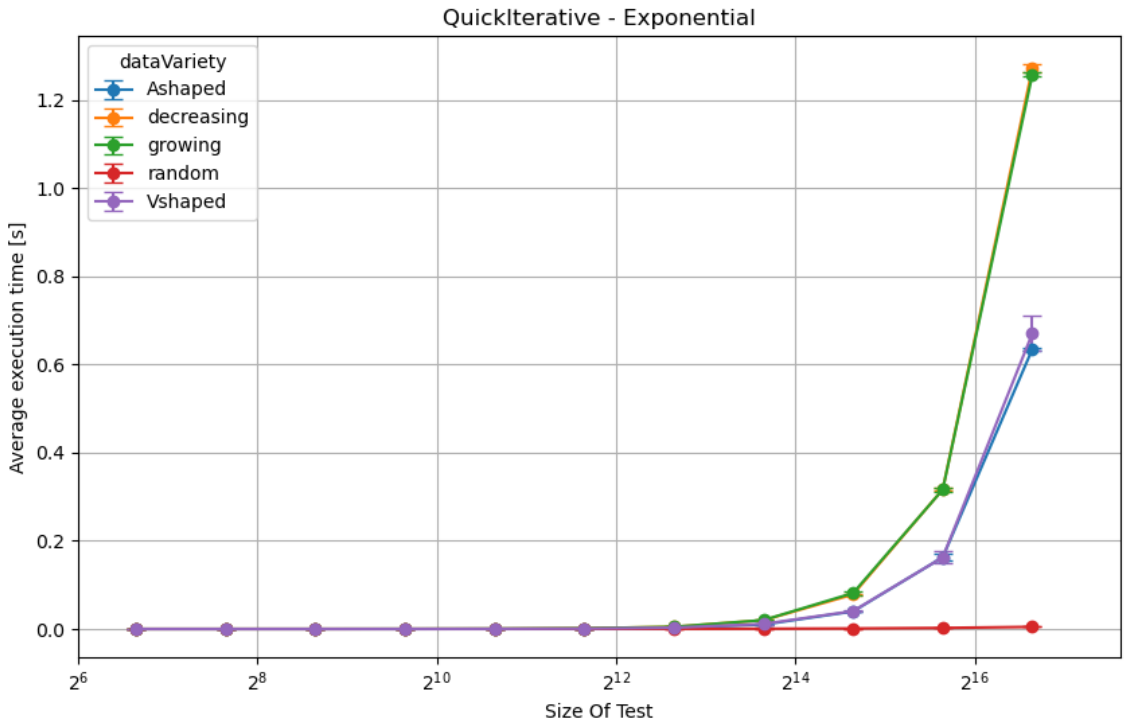


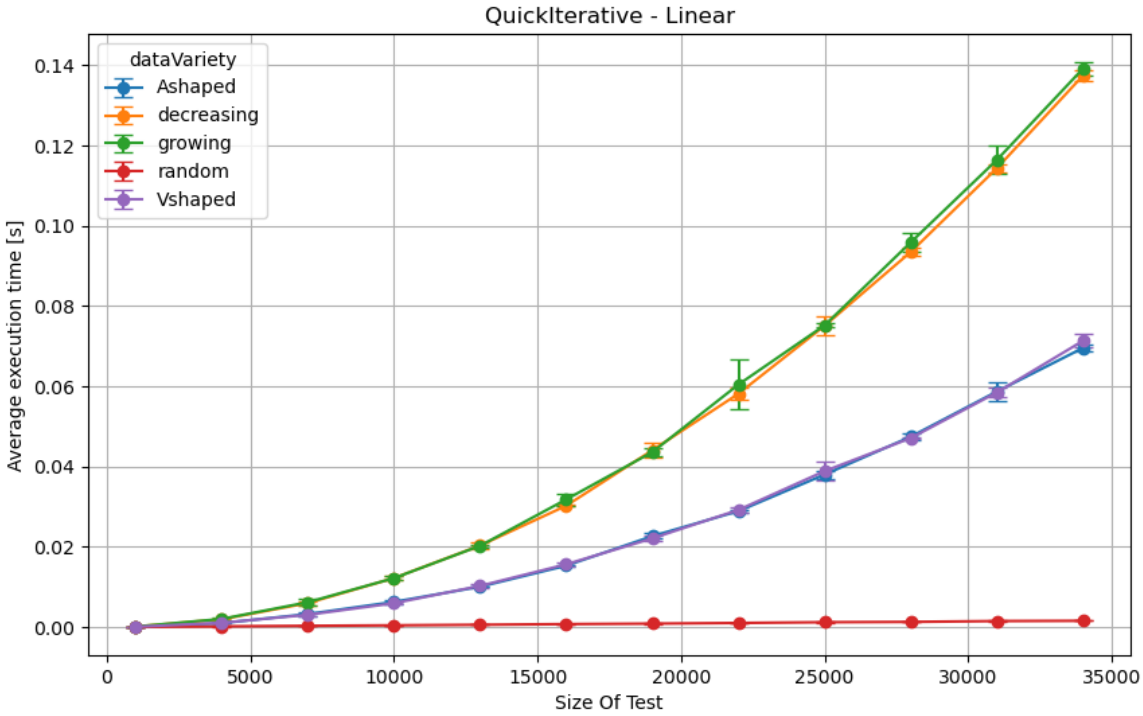

Heap - Exponential
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
2^6
2^8
2^{10}
2^{12}
2^{14}
2^{16}
Size Of Test

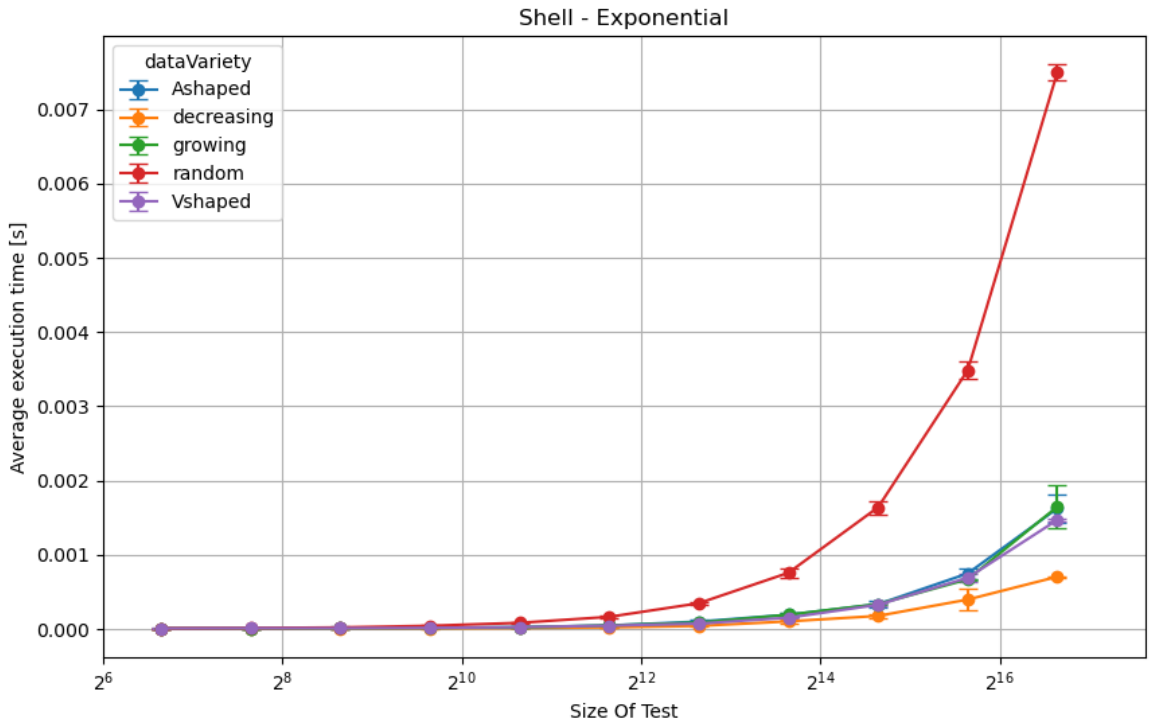

Heap - Linear
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
0.0020
0.0015
0.0010
0.0005
0.0000
0
5000
10000
15000
20000
25000
30000
35000
Size Of Test

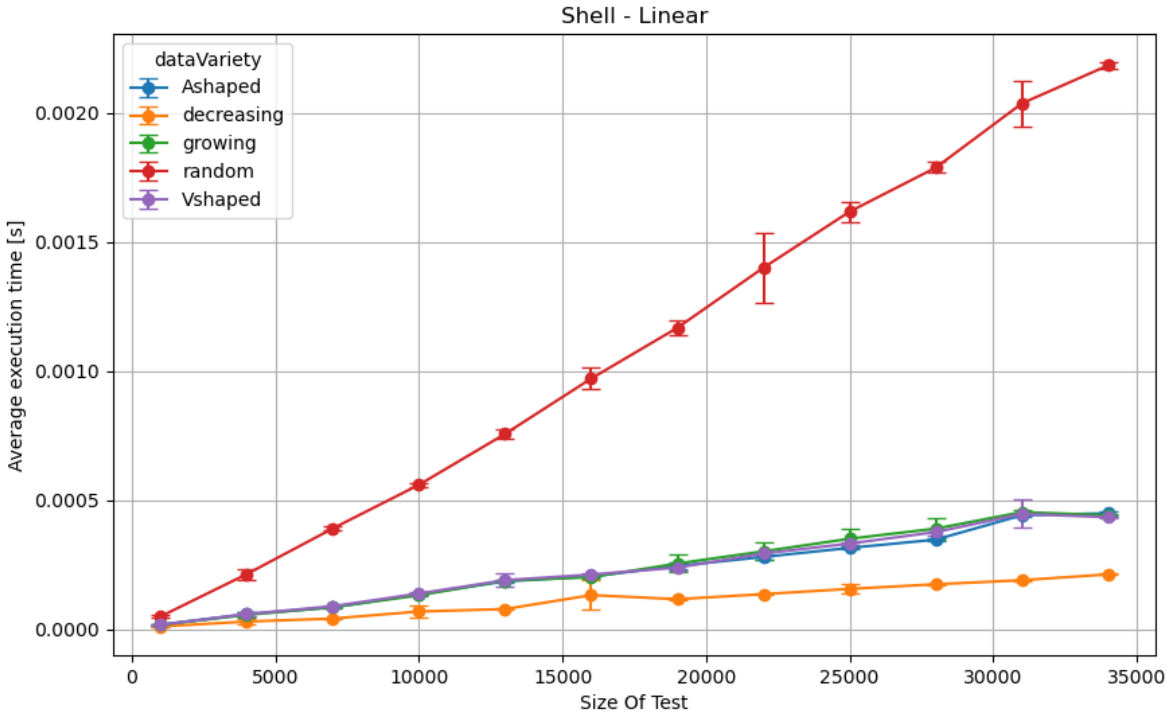

Quick - Exponential
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
Size Of Test
0.0
0.5
1.0
1.5
2.0
2.5
2^6
2^8
2^{10}
2^{12}
2^{14}
2^{16}


Quick - Linear
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
Size Of Test
0.00
0.05
0.10
0.15
0.20
0.25
0
5000
10000
15000
20000
25000
30000
35000

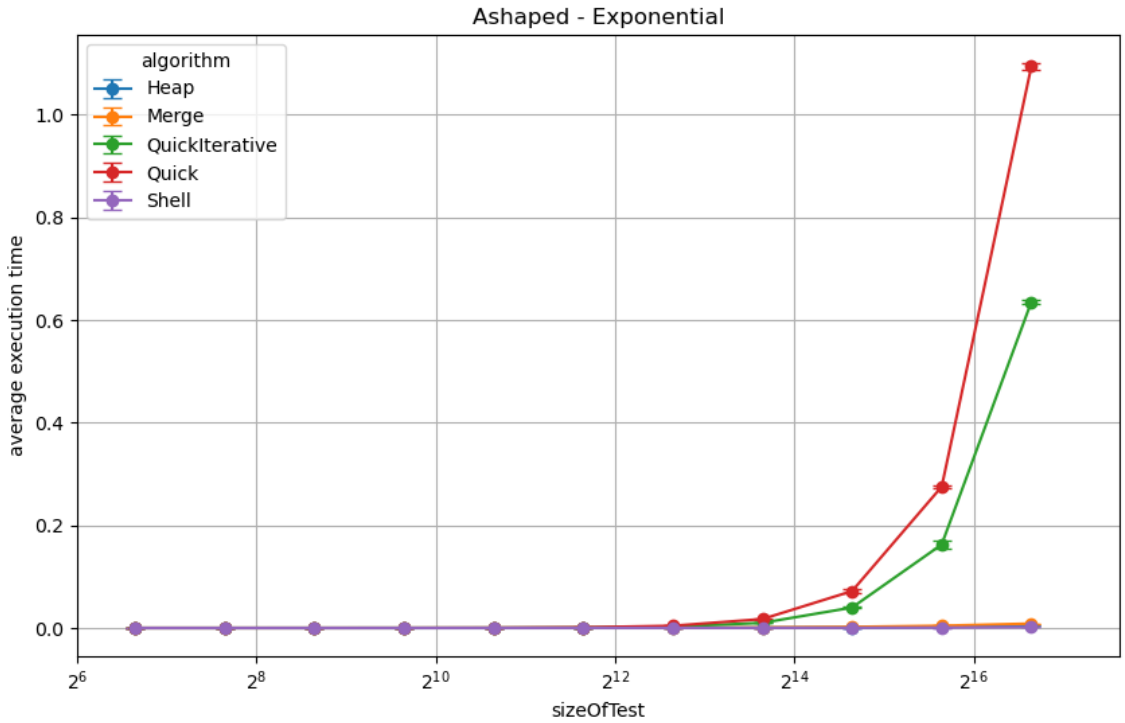

QuickIterative - Exponential
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
0.0
0.2
0.4
0.6
0.8
1.0
1.2
2^6
2^8
2^{10}
2^{12}
2^{14}
2^{16}
Size Of Test

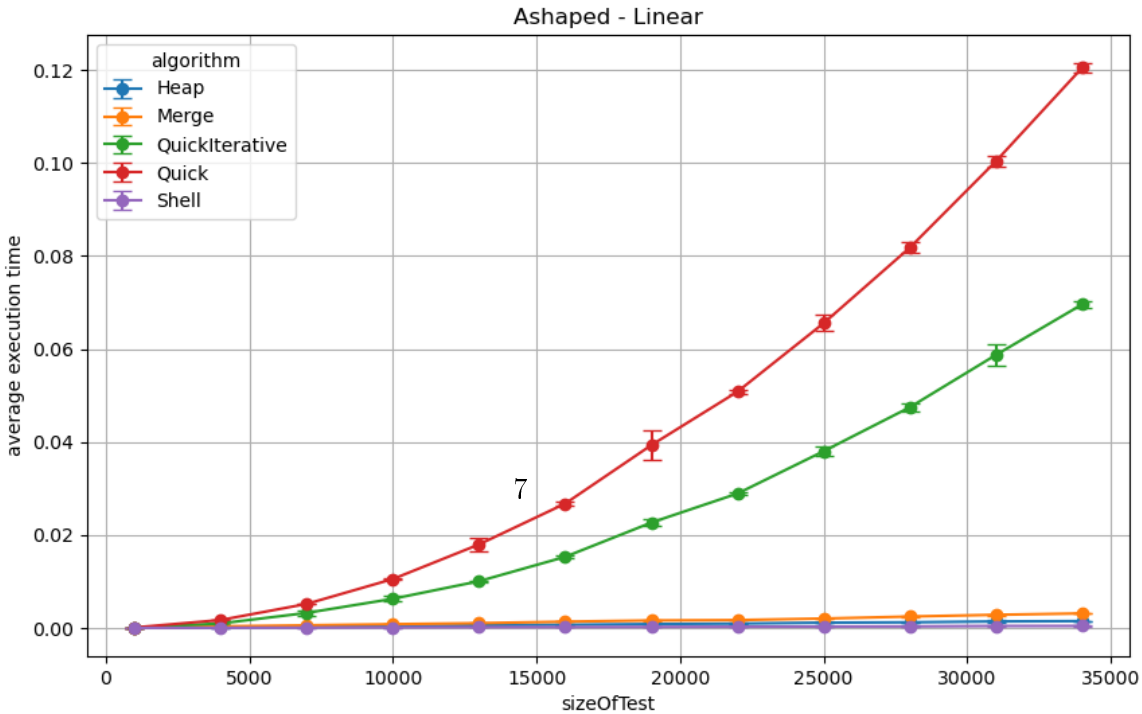

QuickIterative - Linear
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0
5000
10000
15000
20000
25000
30000
35000
Size Of Test

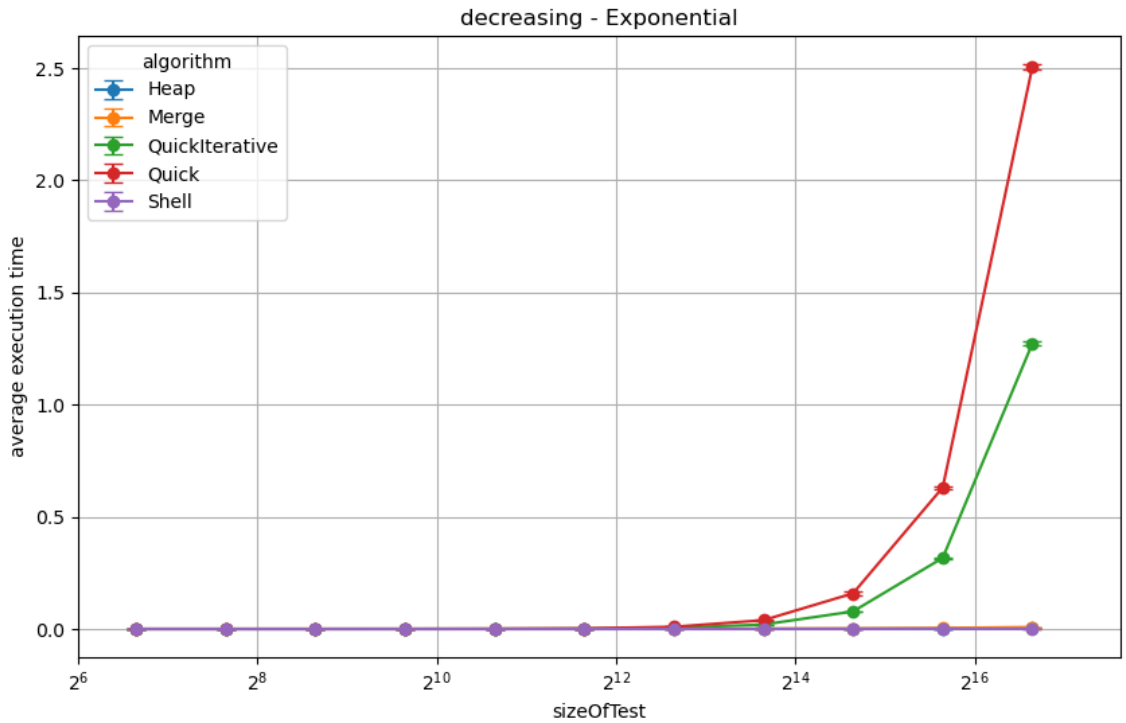

Shell - Exponential
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
0.000
0.001
0.002
0.003
0.004
0.005
0.006
0.007
2^6
2^8
2^{10}
2^{12}
2^{14}
2^{16}
Size Of Test

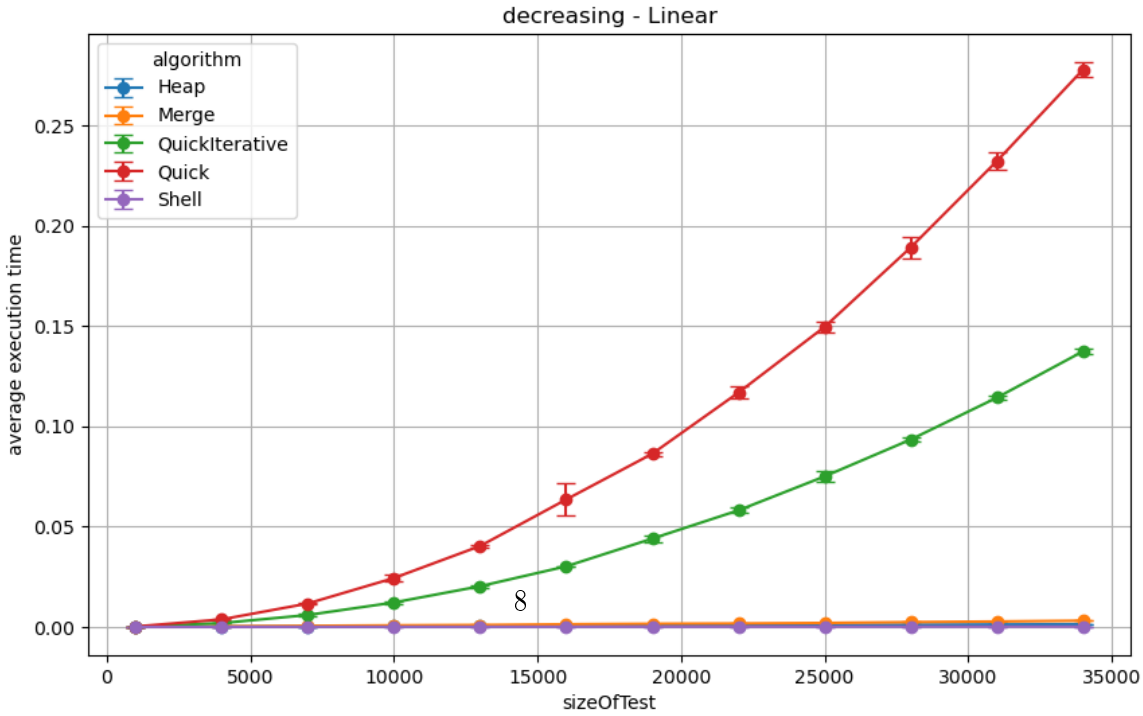

Shell - Linear
dataVariety
Ashaped
decreasing
growing
random
Vshaped
Average execution time [s]
0.0000
0.0005
0.0010
0.0015
0.0020
0
5000
10000
15000
20000
25000
30000
35000
Size Of Test

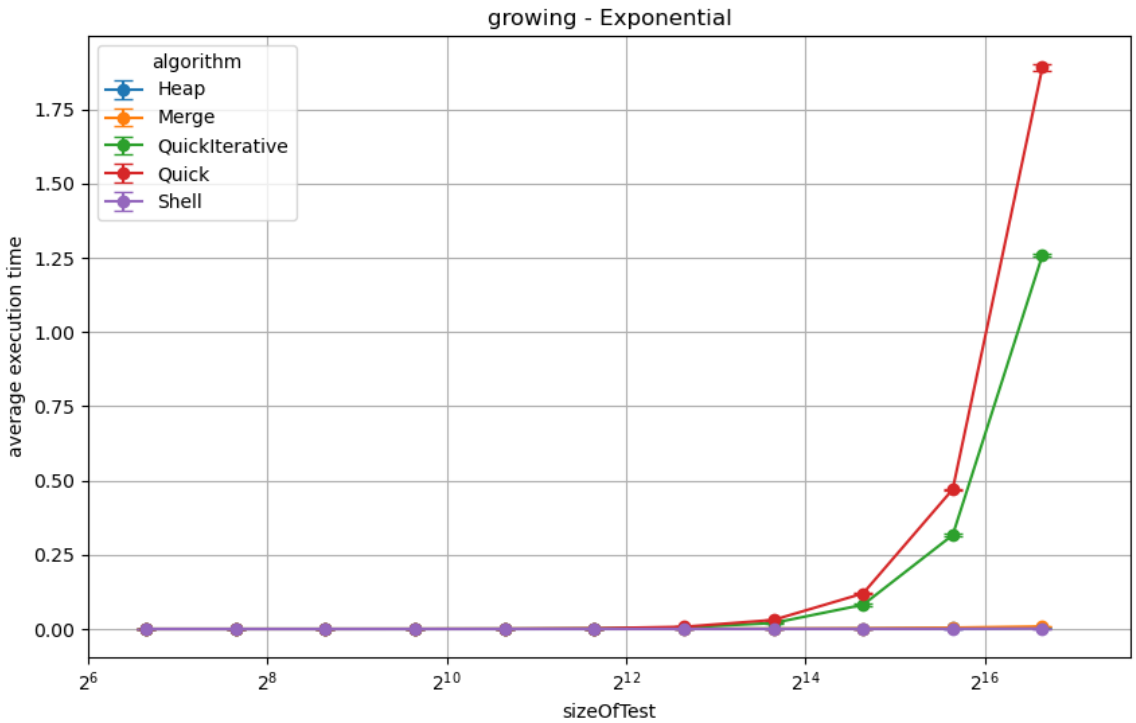
3 Wykresy zależności czasu od liczby elementów według algorytmu.

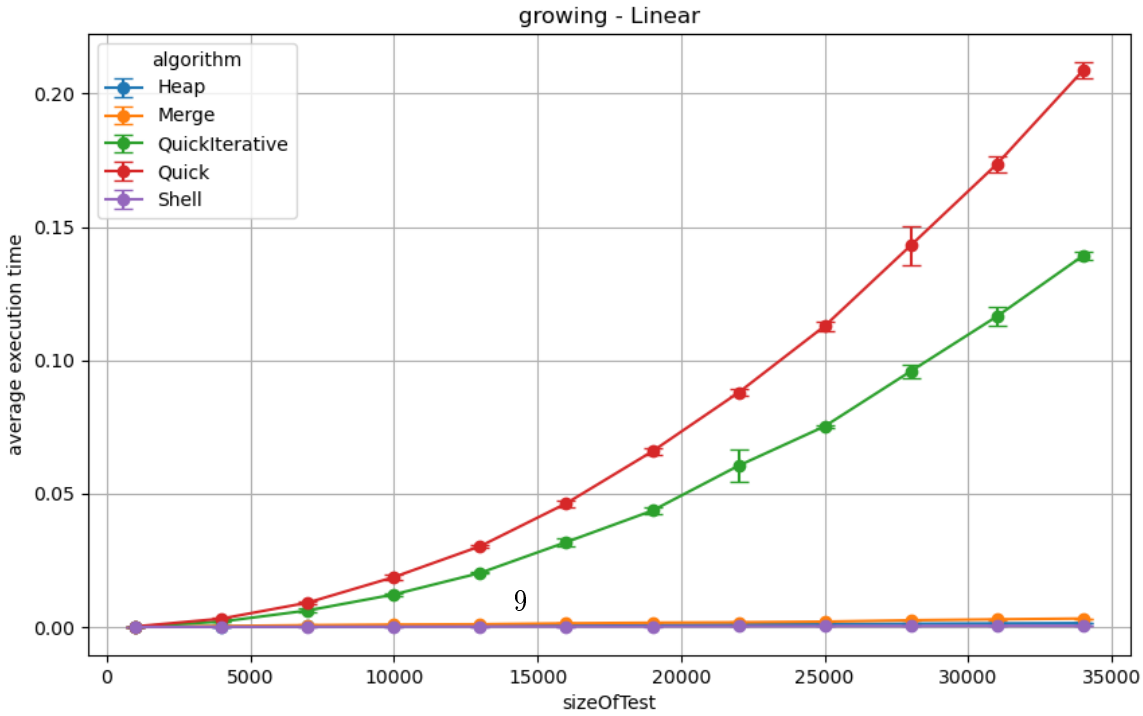


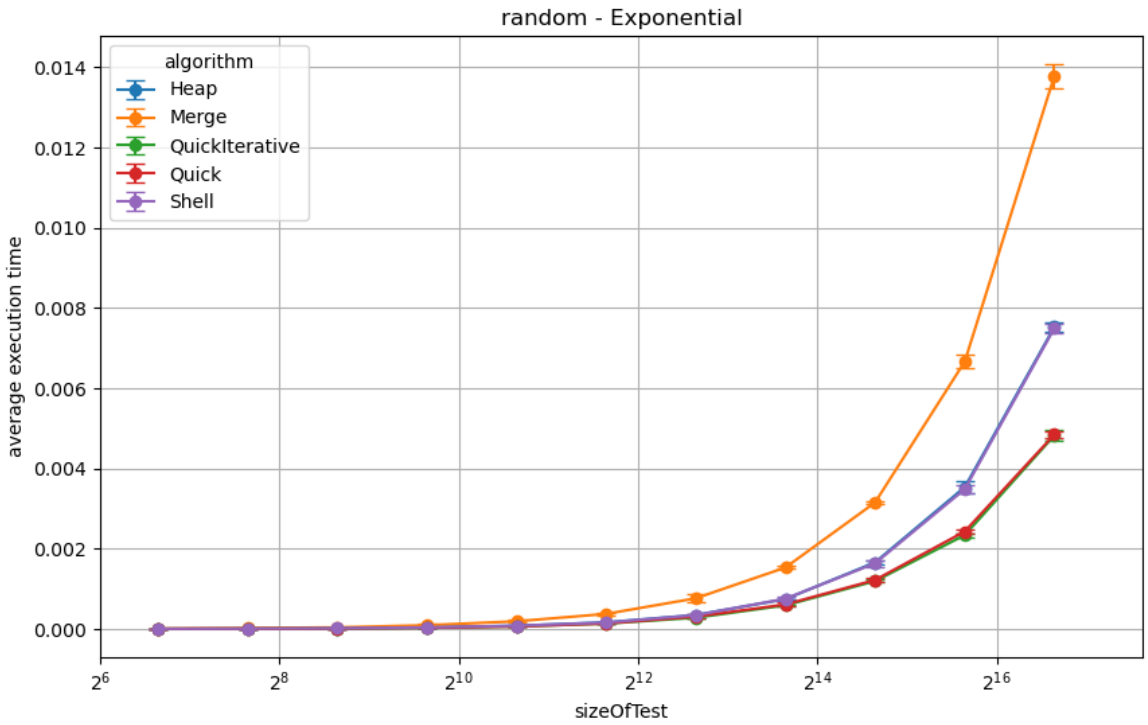


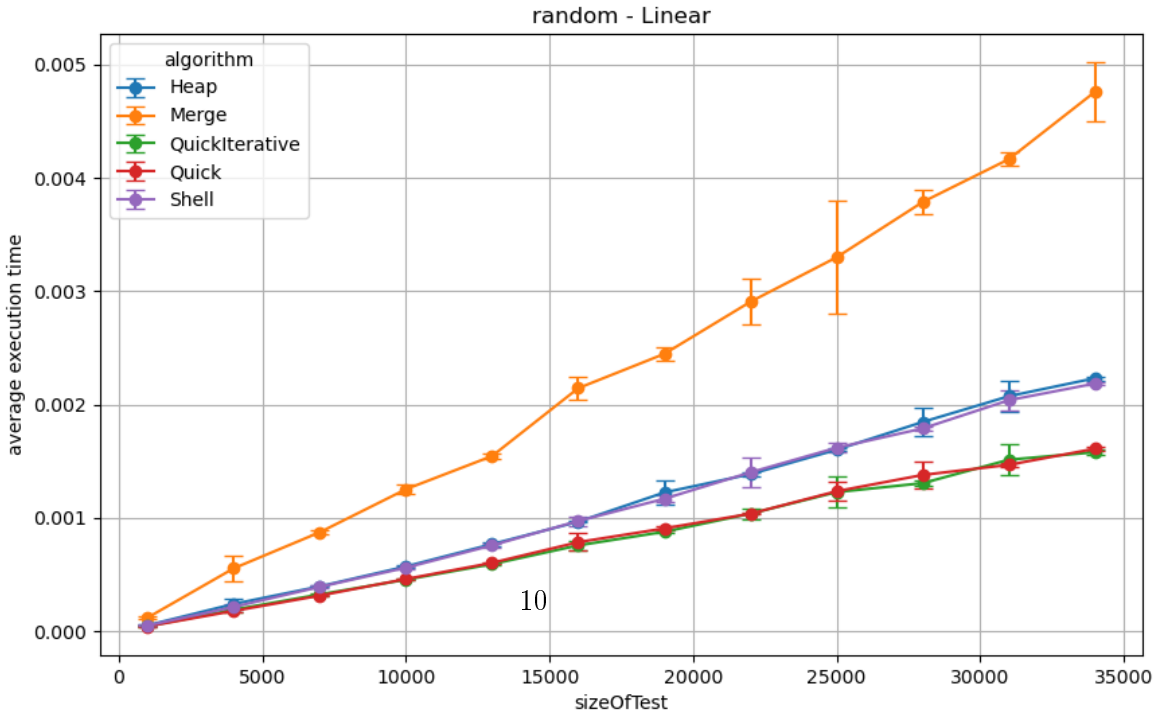

decreasing - Exponential
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
sizeOfTest
0.0
0.5
1.0
1.5
2.0
2.5
2^6
2^8
2^{10}
2^{12}
2^{14}
2^{16}

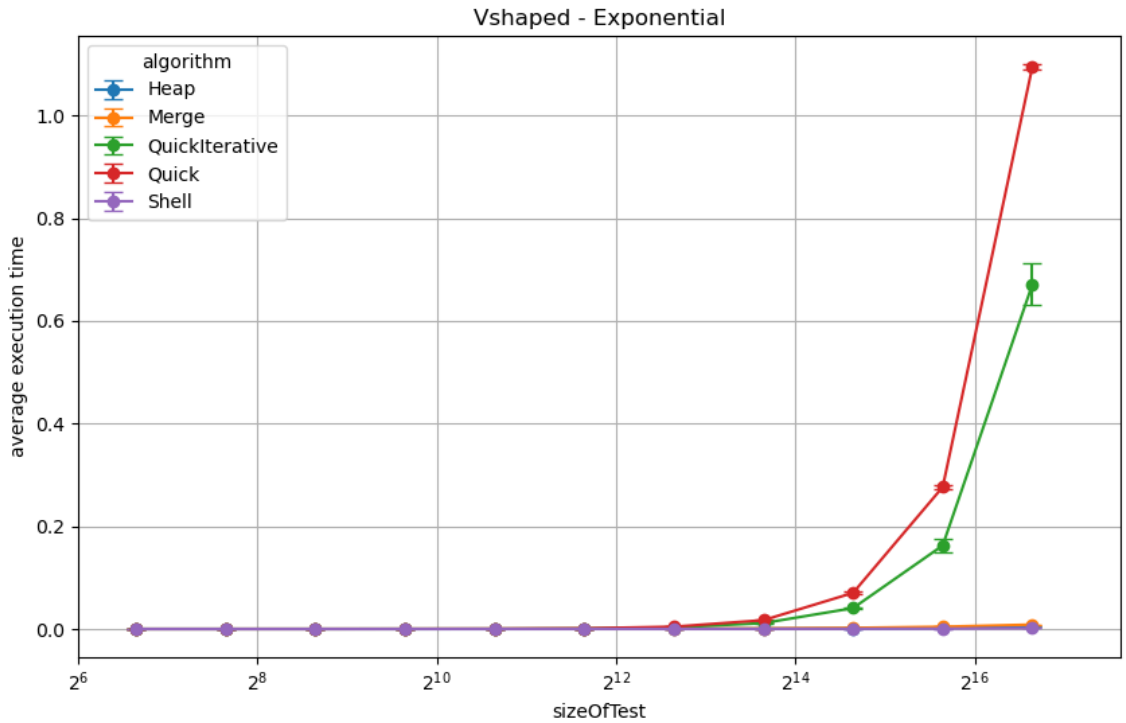

decreasing - Linear
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
sizeOfTest
0.00
0.05
0.10
0.15
0.20
0.25
0
5000
10000
15000
20000
25000
30000
35000

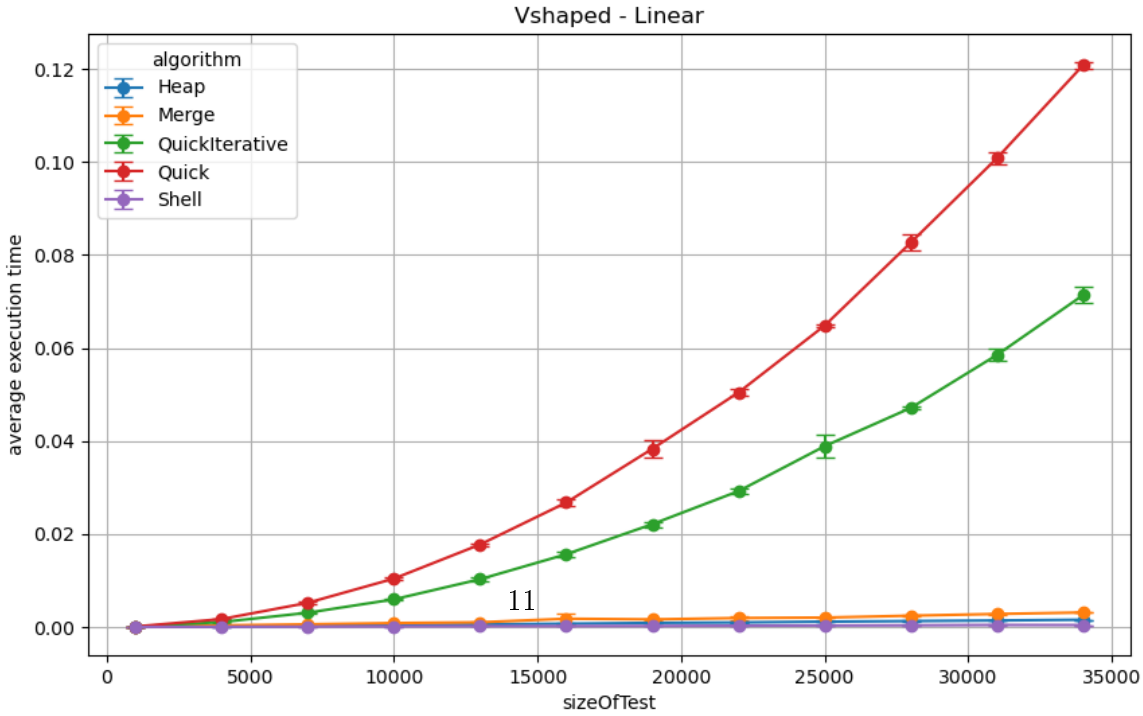

growing - Exponential
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2^6
2^8
2^{10}
2^{12}
2^{14}
2^{16}
sizeOfTest


growing - Linear
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
0.00
0.05
0.10
0.15
0.20
0
5000
10000
15000
20000
25000
30000
35000
sizeOfTest


random - Exponential
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
0.014
0.012
0.010
0.008
0.006
0.004
0.002
0.000
2^6
2^8
2^{10}
2^{12}
2^{14}
2^{16}
sizeOfTest


random - Linear
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
0.005
0.004
0.003
0.002
0.001
0.000
0
5000
10000
15000
20000
25000
30000
35000
sizeOfTest


Vshaped - Exponential
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
sizeOfTest


Vshaped - Linear
algorithm
Heap
Merge
QuickIterative
Quick
Shell
average execution time
sizeOfTest

4 Analiza prównawcza eksperymentów.

1. W eksperymencie pierwszym lepiej widać małe różnicę pomiędzy algorytmami. Przez rosnące liniowo długości sortowanych ciągów, można zauważyć różnicę między algorytmami o tych samych złożonościach obliczeniowych.

2. Przy wzroście wykładniczym długości sortowanych ciągów krzywe czasu średniego przybierają kształty pokrywające się z teoretyczną złożonością obliczeniową. Linie wykresów nie mają odchyleń od swojego trendu.

3. Wzrost wykładniczy daje wykresy z mniejszymi odchyleniami od krzywych teoretycznych. dzięki wykładniczo rosnącym długością ciągów pomiary czasu średniego odbywają się dla bardziej różniących się od siebie długości ciągu co pozwala na eliminację zakłóceń lokalnych i w rezultacie daje wykresy dobrze pokrywające się z teoretyczną złożonością obliczeniową dla danego algorytmu i typu danych.

4. Dla małych długości ciągów różnicę między czasem pracy poszczególnych algorytmów nie są istotne. W znacznej większości przypadków jakiekolwiek różnicę można zaobserwować dopiero dla ciągów mających powyżej 5000 elementów. Jednym wyjątkiem jest tu Merge sort który źle radzi sobie z ciągami wejściowymi typu losowego, w tym przypadku widzimy różnicę już od początku(to jest dla $n<5000$).

5. Odchylenie standardowe ma tendencję rosnącą, którą najlepiej widać dla wykładniczo rosnących n. Natomiast dla eksperymentu pierwszego nie da się wskazać ciągłego trendu z uwagi na zakłócenia lokalne, które są widoczne dla mało różniących się od siebie n. Ale eksperyment drugi daje nam obraz mniej podatny na skrajne wartości lokalne i ukazuje stabilny trend rosnący odchylenia standardowego.

5 Analiza złożoności obliczeniowej.

5.1 Złożoność obliczeniowa dla algorytmów sortowania według rodzaju przypadku:

Algorytm	Optymistyczny	Średni	Pesymistyczny
Shell Sort	$O(n \log n)$	$O(n^{3/2})$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Quick Sort Iteracyjny	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

5.2 Związek między złożonością teoretyczną, liczbą wykonanych operacji i zaobserwowanym czasem działania:

Liczba wykonywanych operacji algorytmu rośnie zgodnie z jego złożonością teoretyczną. Czas wykonania jest w przybliżeniu proporcjonalny do liczby tych operacji, jednak zależy również od czynników zewnętrznych, jak na przykład sprzęt lub obciążenie systemu. Dlatego złożoność teoretyczna nie jest obserwowana bezpośrednio, lecz można ją wnioskować z kształtu wzrostu czasu działania dla rosnących rozmiarów danych.

5.3 Analiza:

Wyniki eksperymentalne w większości potwierdzają analizę teoretyczną. Algorytmy wykazywały podobny do teoretycznego wzrost czasu wraz ze wzrostem n. Quick Sort był najszybszy dla danych losowych, co zgadza się z teorią.

Zarówno rekurencyjny jak i iteracyjny quick sort wykazują eksponencjonalny wzrost dla danych innych niż losowe. Jest to spowodowane nie optymanlnym wyborem pivota, przez co ciąg liczb jest często dzielony w stosunku $1 : n - 1$. Przy innych stragiach wyboru pivota takich jak mediana z trzech (wybieramy dwa elementy i trzeci między nimi, a następnie liczymy ich medianę), quick sort dla danych bliskich posortowaniu wykazuje złożoność obliczoniową bliską $O(n\ log\ n)$.

Dodatkowo wbrew złożoności obliczeniowej merge sort wykazuje wyższe czasy od shell sort. Jest to najpewniej spowodowane tym, że analiza teoretyczna opisuje zachowanie algorytmów dla bardzo dużych wartości n i pomija stałe czynniki oraz szczegóły implementacyjne. W praktyce te czynniki mogą

mieć wpływ na czas działania programu, szczególnie dla mniejszych rozmiarów danych, tak jak okazuje się w naszych eksperymentach. Dla rosnących rozmiarów testów wyniki shell sort okazałyby się gorsze w porównaniu z merge sort.

6 Wnioski końcowe.

Najbardziej efektywnym algorytmem w praktyce okazał się Quick Sort dla danych losowych, osiągając najkrótsze czasy działania. Najbardziej wrażliwy na postać danych wejściowych był również Quick Sort, ponieważ wybór pierwszego elementu jako pivota powodował znaczne pogorszenie wydajności dla danych uporządkowanych lub o specyficznych kształtach.

Porównanie wersji rekurencyjnej i iteracyjnej Quick Sorta wykazało bardzo podobne wyniki, jednak wersja iteracyjna była nieco szybsza, co może wynikać z mniejszego narzutu związanego z wywołaniami rekurencyjnymi.

Wybór pivota miał duży wpływ na wyniki – niekorzystny wybór prowadził do bardzo nierównych podziałów tablicy i pogorszenia złożoności do $O(n^2)$.

Lepszym sposobem prezentacji różnic między algorytmami jest skalowanie wykładnicze, ponieważ pozwala wyraźniej zauważyć różnice w tempie wzrostu czasu działania dla dużych danych.